Anthropic Fellows OA — Complete Prep Pack

Plan, Python, OOP, Inference Engine, Mock OA

August 2026

Contents

3-Day Crash Plan — 1 hour/day

Day	Focus	Material	Output
1	Python syntax refresher	`python_syntax_reference.md`	Drills c1–c8 passing
2	Object-oriented Python	`refresher_day2_oop.md`	Drills c9–c10 passing; class-reading drill
3	Inference engine + mock OA	`inference_engine_primer.md`, `mock_oa/`	Levels 1–2 minimum

Rule for all three days: **type everything, run everything.** Reading alone is worth close to nothing here.

Day 1 — Python syntax (60 min)

Work through `python_syntax_reference.md`. It's a complete sweep — sections are tagged **[OA]** (test-relevant) and **[skim]** (completeness only).

Suggested 60-minute path if you can't do all 20 sections:

Min	Sections	Why
0–10	2 (numbers), 3 (strings)	Division semantics, f-strings, ceiling division
10–25	4 (lists), 5 (tuples/sets), 6 (dicts)	The containers every simulation is built from
25–35	7 (control flow), 8 (comprehensions)	Loop and generator fluency
35–50	**10 (sorting)**, 11 (collections/heapq)	Highest-value 15 minutes in the plan
50–60	16 (copying/mutability), 20 (traps)	The bugs that eat 20 minutes mid-test

Then run `python day1_drills.py` and complete drills **c1–c8**. Anything that takes over 3 minutes marks a section worth rereading. Answers are in `day1_drills_answers.py`.

Sections 12 (files/IO), 15 (typing), 17 (iteration protocol) can wait — they rarely appear in this format.

Day 2 — Object-oriented Python (60 min)

Work through `refresher_day2_oop.md`; use section 14 of the syntax reference as the dense companion cheat sheet.

- **0–30 min: writing classes.** `__init__` and `self`, instance vs. class attributes, `@property` vs. method, dunder methods, dataclasses (the `field(default_factory=...)` rule, the unhashable-by-default rule), `super()` and overriding, `NotImplementedError` stubs.
- **30–40 min: drills c9 and c10** — the `Task` dataclass and the `Pipeline` class. Small on purpose: the goal is producing class scaffolding without pausing to recall syntax.

- **40–60 min: the reading drill.** Open `mock_oa/engine.py` and give yourself 8 timed minutes to write down what each class owns, which members are properties vs. methods, which state you're responsible for updating, and where the mutation risk is. Check yourself against the answers at the end of the Day 2 refresher.

That last drill is the actual exam skill. The Reddit complaints were almost entirely about people skipping it and coding against a misread spec.

Day 3 — Domain + mock OA (60 min)

- **0–15 min: the domain.** `inference_engine_primer.md`, sections 2–5 plus the vocabulary list in section 8. You need enough that *prefill*, *decode*, *KV cache*, *continuous batching*, *load balancing*, and *preemption* don't slow you down in a README. The simulation logic is what's being tested, not the ML — don't go deeper than the primer.
- **15–60 min: the mock, timed at 45 minutes.** `mock_oa/`:
  - First 8 minutes: **read only** — `README.md`, then `engine.py`, then `tests.py`. The tests are the real spec; the README contradicts the code in at least one place, deliberately.
  - Next 12: Level 1. Run `python tests.py` as soon as it's plausible.
  - Remaining 25: Level 2, then Level 3 if it's going well. Skip Level 4.

Afterwards read `SPOILERS.md` and answer one question: *where did the time actually go?* If you have any slack left over the next day, that answer tells you what to redo.

Skip entirely

Graphs, DP, advanced algorithms, LeetCode grinding, PyTorch, GPU/attention math. None of it appears in this format. Queues, heaps, dicts, sorting with explicit tie-breaks, and careful simulation are the whole toolkit.

Test-day protocol

1. Read the README **and the visible tests** first — 10 minutes, no code.
2. Before implementing, write down the tick order and every tie-break rule.
3. Trust tests > class signatures > README prose when they conflict.
4. Get Level 1 green early; it validates the model everything else builds on.
5. Budget roughly 15/20/25/30 minutes per level. Abandon a stuck level and bank partial credit.
6. Expect a later level to invalidate an earlier assumption — keep the admission/selection policy swappable rather than welded into the loop.
7. Print one debug line per tick while developing; delete before submitting.
8. If panic hits: 60 seconds of slow breathing, then restate the current level's contract in one written sentence before touching the keyboard.

Sleep matters more than a fourth hour of prep. Being unhurried in the first ten minutes is the single highest-leverage thing you control.

Python Syntax Reference — Complete Refresher

A full sweep of Python syntax for an experienced engineer who's rusty. Everything is runnable — keep a REPL open. Sections marked **[OA]** are the ones that matter most for a CodeSignal-style systems test; sections marked **[skim]** are completeness, not priority.

Reading order if short on time: 2, 3, 4, 6, 7, 8, 11, 13 → then the rest.

---

1. Program structure & syntax basics

```
# Comments start with #
"""Module docstring — first statement in a file."""

x = 5                       # no declarations, no semicolons, no braces
if x > 3:                   # colon opens a block
    print("big")            # INDENTATION defines the block (4 spaces)
    print("still inside")
print("outside")

# Line continuation
total = (1 + 2 +            # inside brackets: implicit, preferred
         3 + 4)
total = 1 + 2 + \
        3                   # backslash: legal, avoid

a = b = 0                   # chained assignment
a, b = 1, 2                 # tuple assignment
a, b = b, a                 # swap, no temp
x += 1; x -= 1; x *= 2; x //= 2; x **= 2; x %= 3   # no ++ or --

if (n := len(xs)) > 3:      # walrus := assigns inside an expression (3.8+)
    print(n)
```

Naming conventions (enforced by convention, not the compiler): `snake_case` for functions/variables, `PascalCase` for classes, `UPPER_SNAKE` for constants, `_private` by convention, `__mangled` in classes.

Scope: `global x` to rebind a module-level name inside a function; `nonlocal x` to rebind an enclosing function's local. Reading needs neither.

```
counter = 0
def bump():
    global counter
    counter += 1
```

---

2. Numbers, operators, and arithmetic [OA]

```
i = 42                  # int — arbitrary precision, never overflows
f = 3.14                # float — IEEE 754 double
c = 2 + 3j              # complex  [skim]
b = True                # bool IS an int subclass: True + True == 2

7 / 2      # 3.5    — true division, ALWAYS float
7 // 2     # 3      — floor division
```

```
-7 // 2    # -4     — floors toward NEGATIVE INFINITY (not toward zero!)
7 % 3      # 1
-7 % 3     # 2      — result takes the sign of the DIVISOR
2 ** 10    # 1024
divmod(7, 2)          # (3, 1) — quotient and remainder together

abs(-3); round(2.675, 2); round(0.5)   # round() is banker's rounding: 0
int(3.9); int(-3.9)   # 3, -3 — truncates toward zero (unlike //)
float("1.5"); int("42"); int("ff", 16)
```

Ceiling division — memorize both forms:

```
import math
math.ceil(a / b)      # float path; can be wrong for huge ints
-(-a // b)            # exact integer path — prefer this
```

Float comparison is unreliable: `0.1 + 0.2 == 0.3` is `False`. Use `math.isclose(x, y)`. For money/exactness use `decimal.Decimal` or `fractions.Fraction`. **[skim]**

```
math.inf; -math.inf; math.nan          # nan != nan, always
math.floor(x); math.ceil(x); math.sqrt(x); math.log(x, base)
math.gcd(a, b); math.factorial(n); math.comb(n, k)
```

Bitwise: `&  |  ^  ~  <<  >>`. Chained comparison works as in math: `0 <= x < 10` is one expression, evaluating `x` once.

---

3. Strings [OA]

```
s = 'single'; s = "double"          # identical
s = """multi
line"""
s = r"raw\nno escape"               # raw — no backslash processing
b = b"bytes"                        # bytes, not str

# f-strings (3.6+) — your primary formatting and debugging tool
name, val = "t", 3.14159
f"{name}={val}"                     # 't=3.14159'
f"{val:.2f}"                        # '3.14'
f"{val:8.2f}"                       # '    3.14'  width 8, right-aligned
f"{name:<10}|{name:>10}|{name:^10}" # left / right / center pad
f"{42:05d}"                         # '00042'
f"{255:x} {255:b} {255:o}"          # hex, binary, octal
f"{0.256:.1%}"                      # '25.6%'
f"{val=}"                           # 'val=3.14159'  — debug form (3.8+)
f"{ {'a':1}['a'] }"                 # expressions allowed inside
```

Strings are **immutable** — every operation returns a new string.

```
s = "hello world"
len(s); s[0]; s[-1]; s[0:5]; s[::-1]; s[::2]
s.upper(); s.lower(); s.title(); s.capitalize(); s.swapcase()
s.strip(); s.lstrip(); s.rstrip(); s.strip(".,")
s.split(); s.split(","); s.split(",", 1); s.rsplit(); s.splitlines()
",".join(["a", "b"])                # 'a,b'  — join is a STRING method
s.replace("l", "L"); s.replace("l", "L", 1)
s.find("wor")      # 6, or -1 if absent
s.index("wor")     # 6, or raises ValueError
```

```
s.count("l"); "wor" in s
s.startswith("he"); s.endswith(("d", "x"))   # tuple of options allowed
s.isdigit(); s.isalpha(); s.isalnum(); s.isspace(); s.islower()
s.zfill(5); s.center(20, "-"); s.ljust(10); s.rjust(10)
s.removeprefix("hello "); s.removesuffix(" world")     # 3.9+
ord("a"); chr(97)                   # 97, 'a'
"".join(sorted(s))                  # canonical anagram key
```

GOTCHA: building a string in a loop with += is O(n²). Accumulate into a list and `"".join(parts)` at the end.

---

4. Lists [OA]

```
xs = [1, 2, 3]
xs = list(range(5))
xs = [0] * 5                        # [0,0,0,0,0]
grid = [[0] * 3 for _ in range(3)]  # 3x3 — CORRECT
grid = [[0] * 3] * 3                # WRONG: three refs to the SAME row
```

```
xs.append(4)          # add one at end                O(1)
xs.extend([5, 6])     # add many  (xs += [5,6])       O(k)
xs.insert(0, 9)       # insert at index               O(n)
xs.pop()              # remove & return last          O(1)
xs.pop(0)             # remove & return first         O(n) -> use deque
xs.remove(9)          # delete FIRST match by value   O(n), ValueError if absent
del xs[0]; del xs[1:3]
xs.clear()
xs.index(3); xs.index(3, start, end)
xs.count(3)
xs.reverse()          # in place, returns None
xs.sort()             # in place, returns None
sorted(xs)            # new list
xs.copy(); xs[:]      # shallow copy
```

Slicing never raises on out-of-range bounds:

```
xs[1:3]; xs[:2]; xs[2:]; xs[-2:]; xs[::2]; xs[::-1]
xs[1:3] = [9, 9, 9]      # slice assignment can change length
xs[:] = [1, 2]           # replace contents IN PLACE (keeps aliases valid)
```

```
a + b                 # concatenation (new list)
a * 3
3 in a                # O(n) membership — use a set for hot loops
len(a); min(a); max(a); sum(a)
any(x > 2 for x in a); all(x > 0 for x in a)
list(zip(a, b)); list(enumerate(a)); list(reversed(a))
```

---

5. Tuples, sets, frozensets

```
t = (1, 2); t = 1, 2                # parens optional
single = (1,)                       # trailing comma REQUIRED
empty = ()
t[0]; len(t); t + (3,); t * 2       # immutable: no append/assign
```

```
a, b = t                          # unpacking
first, *rest = (1, 2, 3)          # rest == [2, 3]
a, (b, c) = 1, (2, 3)             # nested unpacking
```

Tuples are hashable (if their contents are), so they work as dict keys and set members, and they compare **element by element** — the basis of every multi-key sort. **[OA]**

```
(1, 5) < (1, 9) < (2, 0)          # True
```

Named tuples for readable records: **[skim]**

```
from collections import namedtuple
Point = namedtuple("Point", "x y")
p = Point(1, 2); p.x; p[0]
from typing import NamedTuple
class Point(NamedTuple):
    x: int
    y: int = 0
```

Sets — unordered, unique, O(1) membership:

```
s = {1, 2, 3}; s = set()          # {} is an empty DICT, not a set
s.add(4); s.discard(9)            # discard: no error if absent
s.remove(9)                       # KeyError if absent
s.pop()                           # arbitrary element
a | b   # union            a.union(b)
a & b   # intersection     a.intersection(b)
a - b   # difference
a ^ b   # symmetric difference
a <= b  # subset           a.issubset(b)
frozenset([1, 2])                 # immutable, hashable set
```

Only hashable things go in a set — no lists, and no plain dataclass instances (see the OOP notes).

---

6. Dicts [OA]

```
d = {"a": 1, "b": 2}
d = dict(a=1, b=2)
d = dict([("a", 1)])
d = {k: v for k, v in pairs}
d = dict.fromkeys(["a", "b"], 0)
```

```
d["a"]                # KeyError if missing
d.get("z")            # None
d.get("z", 0)         # default
d.setdefault("k", []).append(1)     # get-or-create then use
d["c"] = 3; del d["c"]
d.pop("a"); d.pop("a", None)
d.popitem()           # removes & returns the LAST inserted pair
d.update({"c": 3}); d |= {"c": 3}   # merge (3.9+);  e = d | other
"a" in d              # checks KEYS, O(1)
len(d); d.clear(); d.copy()

d.keys(); d.values(); d.items()     # dynamic views, not lists
for k in d: ...                     # iterates KEYS
for k, v in d.items(): ...
sorted(d)                           # sorted keys
```

```
sorted(d.items(), key=lambda kv: -kv[1])    # sort by value desc
max(d, key=d.get)                   # key with the largest value
```

Insertion order is guaranteed (3.7+). Keys must be hashable.

GOTCHA: never add or delete keys while iterating a dict — iterate over `list(d)` or `list(d.items())` if you must mutate.

---

7. Control flow [OA]

```
if cond:
    ...
elif other:
    ...
else:
    ...

value = a if cond else b            # ternary

for x in xs: ...
for i in range(n): ...              # range(start, stop, step); stop exclusive
for i, x in enumerate(xs, start=1): ...
for a, b in zip(xs, ys): ...        # stops at the shorter one
for a, b in zip(xs, ys, strict=True): ...   # 3.10+: error on length mismatch
for k, v in d.items(): ...
for _ in range(3): ...              # _ = "I don't need this"

while cond: ...
while True:
    if done: break
    if skip: continue

for x in xs:
    if found: break
else:
    print("loop finished without break")     # for/else — rare but real

pass          # do-nothing placeholder
...           # Ellipsis, also usable as a placeholder
```

Boolean logic uses words and short-circuits:

```
a and b; a or b; not a
x = val or "default"        # falls back when val is falsy (careful with 0!)
0 <= x < 10                 # chained comparison
a is b; a is not b          # identity, not equality
x in xs; x not in xs
```

`match` (3.10+) — structural pattern matching: **[skim]**

```
match command.split():
    case ["go", direction]:
        move(direction)
    case ["quit"] | ["exit"]:
        stop()
    case {"type": "req", "id": rid}:      # dict pattern
        handle(rid)
```

```
    case Point(x=0, y=y):                # class pattern
        ...
    case _:
        unknown()
```

---

8. Comprehensions & generators [OA]

```
[x * 2 for x in xs]                          # list
[x for x in xs if x > 0]                     # filter
[x if x > 0 else 0 for x in xs]              # ternary BEFORE the `for`
[(i, j) for i in range(3) for j in range(3)] # nested loops, outer first
[[c for c in row] for row in grid]           # nested comprehension
{k: v for k, v in pairs}                     # dict
{v: k for k, v in d.items()}                 # invert a dict
{x % 3 for x in xs}                          # set
(x * 2 for x in xs)                          # GENERATOR — lazy, one pass
sum(r.tokens for r in reqs)                  # bare genexp as sole argument
```

Generators are lazy and memory-cheap; they can only be consumed once.

```
def countdown(n):
    while n > 0:
        yield n            # produces a value, suspends here
        n -= 1
    return                 # StopIteration

g = countdown(3)
next(g); next(g)
list(countdown(3))         # [3, 2, 1]

def flatten(nested):
    for sub in nested:
        yield from sub     # delegate to another iterable
```

`itertools` — the standard toolbox: **[skim]**

```
from itertools import (count, cycle, repeat, chain, islice, product,
                       permutations, combinations, groupby, accumulate,
                       pairwise, takewhile, dropwhile)
chain([1,2], [3])            # 1 2 3
islice(count(), 5)           # first 5 of an infinite counter
product([1,2], repeat=2)     # (1,1) (1,2) (2,1) (2,2)
combinations([1,2,3], 2)     # (1,2) (1,3) (2,3)
accumulate([1,2,3])          # 1 3 6  — running totals
pairwise([1,2,3])            # (1,2) (2,3)      3.10+
groupby(sorted(xs, key=f), key=f)   # requires pre-sorting!
```

---

9. Functions

```
def f(a, b=10, *args, **kwargs):
    """Docstring."""
    return a + b                    # bare `return` / falling off end -> None
```

```
def g(): return 1, 2                # returns a tuple
x, y = g()

f(1, 2)                             # positional
f(a=1, b=2)                         # keyword
f(*[1, 2]); f(**{"a": 1, "b": 2})   # unpack into arguments

def h(a, /, b, *, c):               # a: positional-only; c: keyword-only
    ...
```

GOTCHA — mutable default arguments are evaluated ONCE at def time:

```
def bad(x, acc=[]):        # NEVER
    acc.append(x); return acc
bad(1); bad(2)             # [1, 2] — same list

def good(x, acc=None):
    acc = [] if acc is None else acc
```

Lambdas — single expression, no statements:

```
key = lambda r: (r.arrival, r.id)
sorted(xs, key=lambda r: -r.size)
```

Closures and decorators: **[skim for the OA, but recognize them]**

```
def make_counter():
    n = 0
    def inc():
        nonlocal n
        n += 1
        return n
    return inc

import functools

def logged(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        print(f"calling {fn.__name__}")
        return fn(*args, **kwargs)
    return wrapper

@logged                    # equivalent to: f = logged(f)
def f(): ...

@functools.lru_cache(maxsize=None)     # memoization, free
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
```

`functools.reduce`, `partial`, and `cmp_to_key` round out the module.

---

10. Sorting — the most test-relevant topic [OA]

```
xs.sort()                                  # in place, returns None
ys = sorted(xs)                            # new list
sorted(xs, reverse=True)
```

```
sorted(xs, key=lambda r: r.arrival)                 # single key
sorted(xs, key=lambda r: (r.arrival, r.id))         # multi-key
sorted(xs, key=lambda r: (-r.priority, r.arrival, r.id))   # mixed direction
sorted(xs, key=len)                                 # any callable
sorted(d.items(), key=lambda kv: (-kv[1], kv[0]))   # by value desc, key asc
```

Rules to internalize:

1. A tuple key compares left to right — first field decides, later fields break ties.
2. Negate a numeric field to make it descending. `reverse=True` flips **every** field, which is usually not what a spec asks for.
3. For descending on non-numerics, sort twice (stable sort preserves the earlier order) or use `functools.cmp_to_key`.
4. `sorted` is **stable**: equal keys keep input order. Convenient, but when the spec states a tie-break, write it into the key explicitly.

Selection with the same discipline:

```
min(xs); max(xs)
min(xs, key=lambda r: (r.load, r.index))
min(range(len(loads)), key=lambda i: (loads[i], i))   # ties -> lowest index
max(items, key=lambda r: (-r.priority, r.id))         # ties -> highest id
```

Binary search on a sorted list:

```
import bisect
bisect.bisect_left(xs, v)      # leftmost insertion point
bisect.bisect_right(xs, v)     # rightmost
bisect.insort(xs, v)           # insert keeping order  (O(n) move)
```

---

11. collections, heapq, and friends [OA]

```
from collections import deque, defaultdict, Counter, OrderedDict, ChainMap

q = deque([1, 2, 3])
q.append(4); q.appendleft(0)        # O(1) both ends
q.pop(); q.popleft()                # O(1) both ends
q[0]; len(q); q.rotate(1); deque(maxlen=5)

d = defaultdict(list);  d[k].append(v)      # missing key -> []
d = defaultdict(int);   d[k] += 1           # missing key -> 0
d = defaultdict(set);   d[k].add(v)

c = Counter("aabbbc")               # {'b':3,'a':2,'c':1}
c.most_common(2); c["z"]            # 0 for missing, no KeyError
c.update("ab"); c1 + c2; c1 - c2; c.total()
```

```
import heapq
h = []
heapq.heappush(h, (priority, tiebreak_id, payload))
priority, _, payload = heapq.heappop(h)      # SMALLEST first
h[0]                                         # peek
heapq.heapify(xs)                            # O(n), in place
heapq.heappushpop(h, item); heapq.heapreplace(h, item)
heapq.nsmallest(3, xs, key=...); heapq.nlargest(3, xs, key=...)
```

Max-heap trick: push `-value`, or push `(-priority, id, obj)`. **Always include a determinis-**

tic tiebreak field before any non-comparable payload, or Python will try to compare the payloads and crash.

---

12. Files, JSON, and I/O [skim for the OA]

```
with open("f.txt") as fh:            # context manager closes it for you
    text = fh.read()
    # fh.readline(); fh.readlines(); for line in fh: ...

with open("f.txt", "w") as fh:       # 'r' 'w' 'a' 'x', add 'b' for binary
    fh.write("hi\n")
    fh.writelines(["a\n", "b\n"])

import json
json.dumps(obj, indent=2, sort_keys=True); json.loads(s)
json.dump(obj, fh); json.load(fh)

import csv
reader = csv.DictReader(fh); writer = csv.DictWriter(fh, fieldnames=[...])

from pathlib import Path
p = Path("dir") / "f.txt"
p.exists(); p.read_text(); p.write_text("x"); p.stem; p.suffix; p.parent
list(Path(".").glob("**/*.py"))

import sys
sys.argv; sys.exit(1); print("err", file=sys.stderr)
input("prompt: ")
```

---

13. Exceptions [OA]

```
try:
    risky()
except (KeyError, IndexError) as e:
    print(type(e).__name__, e)
except Exception as e:               # never bare `except:`
    raise                            # re-raise, preserving traceback
else:
    print("no exception occurred")
finally:
    cleanup()                        # always runs

raise ValueError("message")
raise ValueError("msg") from original_error
assert cond, "message"               # AssertionError; stripped under -O

class MyError(Exception):
    pass
```

Common built-ins: `ValueError`, `TypeError`, `KeyError`, `IndexError`, `AttributeError`, `ZeroDivisionError`, `StopIteration`, `NotImplementedError` (the stub marker you'll be replacing in the OA).

EAFP is idiomatic Python — try it and catch the failure, rather than checking first:

```
try:                    # EAFP: Easier to Ask Forgiveness than Permission
    v = d[k]
except KeyError:
    v = default
v = d.get(k, default)   # ...though here the direct method is better still
```

---

14. Classes — full syntax [OA] (detail in the Day 2 refresher)

```
class Worker:
    """Docstring."""
    registry = []                   # CLASS attribute — shared by all!

    def __init__(self, index):
        self.index = index          # instance attribute
        self.running = []           # mutable state belongs here

    def load(self):                 # instance method
        return len(self.running)

    @property
    def is_full(self):              # accessed WITHOUT parens
        return self.load() >= 4

    @is_full.setter
    def is_full(self, value): ...

    @staticmethod
    def helper(x):                  # no self — just namespaced
        return x * 2

    @classmethod
    def from_config(cls, cfg):      # alternate constructor
        return cls(cfg.index)

    def __repr__(self):  return f"Worker({self.index})"
    def __str__(self):   return f"W{self.index}"
    def __eq__(self, o): return self.index == o.index
    def __hash__(self):  return hash(self.index)
    def __lt__(self, o): return self.index < o.index
    def __len__(self):   return len(self.running)
    def __iter__(self):  return iter(self.running)
    def __contains__(self, x): return x in self.running
    def __getitem__(self, i):  return self.running[i]
    def __bool__(self):  return bool(self.running)
    def __call__(self, x): ...
    def __enter__(self); def __exit__(self, *exc)     # context manager
```

Inheritance:

```
class Base:
    def pick(self): raise NotImplementedError

class Fifo(Base):
    def __init__(self, cfg):
        super().__init__()
        self.cfg = cfg
```

```
    def pick(self): return self.queue[0]

isinstance(x, Base); issubclass(Fifo, Base)

from abc import ABC, abstractmethod
class Scheduler(ABC):
    @abstractmethod
    def pick(self): ...              # cannot instantiate without overriding
```

Dataclasses:

```
from dataclasses import dataclass, field, asdict, replace

@dataclass(frozen=False, order=False, slots=True)
class Request:
    id: int
    arrival: int
    tokens: int = 0
    log: list = field(default_factory=list)     # mutable default REQUIRES this
    _cache: dict = field(default_factory=dict, repr=False, compare=False)

    @property
    def reserved(self): return self.tokens * 2

    def __post_init__(self):         # runs after generated __init__
        assert self.tokens >= 0

asdict(r); replace(r, tokens=5)      # convert / copy-with-changes
```

`@dataclass` generates `__init__`, `__repr__`, `__eq__`. Because `eq=True` it sets `__hash__ = None` → **instances are unhashable**; use `frozen=True` if you need them in a set. `order=True` generates `<` etc.

Enums: **[skim]**

```
from enum import Enum, auto
class State(Enum):
    WAITING = auto()
    RUNNING = auto()
State.WAITING.name; State.WAITING.value; list(State)
```

---

15. Modules, imports, and typing

```
import math
import numpy as np
from collections import deque
from collections import deque as dq
from mypkg.mod import thing
from . import sibling          # relative import inside a package

if __name__ == "__main__":     # runs only when executed directly
    main()
```

```
from typing import Optional, Any, Callable, Iterable, Iterator, Union

def f(x: int, ys: list[str], d: dict[str, int]) -> Optional[int]: ...
```

```
def g(cb: Callable[[int], str], xs: Iterable[int]) -> Iterator[int]: ...
x: int | None = None           # 3.10+ union syntax
```

Type hints are never enforced at runtime — they're documentation plus tooling.

---

16. Copying, identity, and mutability [OA]

The concept that causes the most confusion coming from other languages: **names are references, assignment never copies.**

```
a = [1, 2]; b = a; b.append(3);  a        # [1, 2, 3] — same object
a is b                                     # True

import copy
b = a[:]  or  a.copy()  or  list(a)        # shallow — nested objects shared
b = copy.deepcopy(a)                       # fully independent

grid = [[0] * 3] * 3                       # three references to ONE row
grid[0][0] = 1; grid                       # [[1,0,0],[1,0,0],[1,0,0]]
```

Function arguments are passed by reference-to-object: mutating a list argument affects the caller; rebinding the name inside the function does not.

```
def f(xs):
    xs.append(1)      # visible to the caller
    xs = [9]          # NOT visible — rebinds the local name only
```

Immutable: `int, float, str, bytes, tuple, frozenset, bool, None`. Mutable: `list, dict, set, bytearray`, and most class instances.

---

17. Iteration protocol & useful built-ins

```
iter(xs); next(it); next(it, default)

class Countdown:
    def __init__(self, n): self.n = n
    def __iter__(self): return self
    def __next__(self):
        if self.n <= 0: raise StopIteration
        self.n -= 1
        return self.n + 1
```

```
len, sum, min, max, abs, round, sorted, reversed, enumerate, zip
any, all, map, filter, range, type, isinstance, id, hash, repr
int, float, str, bool, list, tuple, dict, set, frozenset
divmod, pow, format, print, input, open, dir, vars, getattr, setattr
```

`map`/`filter` are rarely idiomatic — prefer comprehensions:

```
list(map(str, xs))        ==  [str(x) for x in xs]
list(filter(None, xs))    ==  [x for x in xs if x]
```

---

18. Debugging techniques for a timed test [OA]

```
print(f"{t=} {reserved=} running={[r.id for r in running]}")   # f-string debug
assert reserved >= 0, f"negative memory at t={t}"

import pprint; pprint.pprint(complex_structure)
print(vars(obj))          # instance __dict__ — all attributes at once
print(dir(obj))           # every available member — great for unfamiliar APIs
help(obj.method)          # docstring, offline

import traceback; traceback.print_exc()
breakpoint()              # drops into pdb: n(ext) s(tep) c(ont) p(rint) q(uit)
```

In a tick-based simulation, printing one line per tick showing the clock, queue, running set, and memory is the fastest possible way to find an off-by-one. Write it early, delete it before submitting.

---

19. Complexity cheat sheet [OA]

Operation	list	deque	dict/set	heapq
index / key lookup	O(1)	O(1) ends	O(1)	—
append / push	O(1)*	O(1)	O(1)	O(log n)
pop from end	O(1)	O(1)	—	O(log n)
pop from front	**O(n)**	O(1)	—	—
insert / delete middle	O(n)	O(n)	O(1) by key	—
`in` membership	**O(n)**	O(n)	O(1)	—
min element	O(n)	O(n)	O(n)	**O(1)**
sort	O(n log n)	—	—	—

* amortized. The two decisions that matter in practice: use `deque` when you pop from the front, and use a `set`/`dict` when you test membership in a loop.

---

20. Twelve traps, collected [OA]

1. `a = b` doesn't copy; `b.append()` mutates both.
2. `[[0]*3]*3` aliases rows — use a comprehension.
3. Mutable default arguments persist across calls.
4. `list: list = []` in a dataclass raises — use `field(default_factory=list)`.
5. Mutating a list while iterating it silently skips elements.
6. `-7 // 2 == -4`, and `int(a/b) ≠ a // b` for negatives.
7. `0.1 + 0.2 != 0.3` — use `math.isclose`.
8. `xs.sort()` returns `None`; `sorted(xs)` returns the list.
9. `reverse=True` flips all sort keys, not just one — negate instead.
10. `@property` members are accessed without parentheses; calling one raises a confusing `TypeError` far from the real mistake.
11. `@dataclass` instances are unhashable unless `frozen=True`.
12. Heap pushes of bare objects crash on ties — push tuples with an id.

Practice

`day1_drills.py` — 10 exercises covering the sections marked **[OA]**. Run it, fill in the TODOs, check against `day1_drills_answers.py`.

Day 2 — Object-Oriented Python (60 min)

The test hands you a pre-built class hierarchy and asks you to extend it. So this session has two halves: writing classes, and **reading** someone else's fast. Type everything.

---

1. Classes from zero (12 min)

```python
class Worker:
    """Docstring."""

    def __init__(self, index, capacity):   # constructor
        self.index = index                  # instance attributes
        self.capacity = capacity
        self.running = []                   # created fresh per instance

    def add(self, req):                     # `self` is ALWAYS explicit
        self.running.append(req)

    def load(self):
        return sum(r.size for r in self.running)

    def __repr__(self):                     # what print() shows
        return f"Worker({self.index}, load={self.load()})"


w = Worker(0, 100)
w.add(req)
print(w.load())        # method — needs parens
```

Key differences from other languages:

- **No `new`** — call the class: `Worker(0, 100)`.
- **`self` is explicit** in every method definition, but not at the call site.
- **No private/public.** A leading underscore (`_cache`) means "internal, by convention." Two underscores (`__x`) triggers name mangling — rarely used.
- **No overloading.** One `__init__` per class; use default arguments instead.
- Attributes are created by assignment — `self.anything = 1` just works, which means typos silently create new attributes rather than erroring.

GOTCHA — class attributes are shared across all instances:

```python
class Bad:
    items = []                # ONE list for the whole class
    def __init__(self):
        self.count = 0        # per instance — correct

a, b = Bad(), Bad()
a.items.append(1)
b.items                       # [1]  — surprise
```

Rule: anything mutable belongs in `__init__`, assigned to `self`.

---

2. Dunder methods worth knowing (8 min)

```
class Req:
    def __init__(self, id): self.id = id
    def __repr__(self):  return f"Req({self.id})"      # debugging
    def __eq__(self, o): return self.id == o.id        # ==
    def __hash__(self):  return hash(self.id)          # needed for set/dict
    def __lt__(self, o): return self.id < o.id         # enables sorting
    def __len__(self):   return 1                      # len()
```

GOTCHA: defining `__eq__` without `__hash__` makes the class **unhashable** — it can no longer go in a `set` or be a dict key. And without `__lt__`, pushing objects into a heap crashes on ties. That's why you push `(key, id, obj)` tuples instead of bare objects.

---

3. Properties (8 min)

A method that's accessed like an attribute — very common in the codebases these tests give you.

```
class Request:
    def __init__(self, prompt_tokens, max_new_tokens):
        self.prompt_tokens = prompt_tokens
        self.max_new_tokens = max_new_tokens

    @property
    def reserved(self):                    # computed on every access
        return self.prompt_tokens + self.max_new_tokens

r.reserved        # 12   — NO parentheses
r.reserved()      # TypeError: 'int' object is not callable
```

When you read an unfamiliar class, **check whether each member is a `@property` or a method** before you call it. Getting this wrong produces a confusing error at a distance and is a classic 10-minute time sink.

---

4. Dataclasses — what you'll mostly see (12 min)

Boilerplate-free classes for holding data. `__init__`, `__repr__`, and `__eq__` are generated for you.

```
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class Request:
    id: int                                  # required, positional
    arrival: int
    prompt_tokens: int
    tokens_generated: int = 0                # defaults AFTER non-defaults
    finished_at: Optional[int] = None
    log: list = field(default_factory=list)  # mutable default -> factory!

    @property
    def done(self):
        return self.finished_at is not None

    def fresh(self):                         # explicit reset copy
```

```
        return Request(self.id, self.arrival, self.prompt_tokens)

r = Request(1, 0, 8)
r = Request(id=1, arrival=0, prompt_tokens=8)   # keywords also work
print(r)     # Request(id=1, arrival=0, ...)  — free __repr__
```

Three rules that cause real bugs:

1. `log: list = []` raises `ValueError` in a dataclass — you must use `field(default_factory=list)`. (Plain classes let the same mistake pass silently, which is worse.)
2. Fields with defaults must come after fields without.
3. `@dataclass` sets `eq=True`, which sets `__hash__ = None` → **unhashable**. Use `@dataclass(frozen=True)` for a hashable, immutable version, or key collections by `.id`.

Mutability matters for this test: dataclass instances are mutable by default, so a function that mutates the requests it's given will corrupt a second run over the same objects. That's exactly why a `fresh()` helper exists.

---

5. Inheritance, briefly (5 min)

```
class Scheduler:
    def __init__(self, config): self.config = config
    def pick(self, queue): raise NotImplementedError    # abstract-ish

class FifoScheduler(Scheduler):
    def __init__(self, config, quantum):
        super().__init__(config)         # call the parent constructor
        self.quantum = quantum
    def pick(self, queue):               # override
        return queue[0]

isinstance(s, Scheduler)   # True
```

You rarely need deep hierarchies in these tests, but you must recognize `super()`, overriding, and `NotImplementedError` stubs — the stubs are literally what you're asked to fill in.

---

6. Reading an unfamiliar OOP codebase fast (10 min) — DO THIS

This is the actual exam skill. Open `mock_oa/engine.py` and give yourself **8 minutes** to answer, in writing:

1. What classes exist, and what does each one *own* (which data)?
2. For each member: is it a field, a `@property`, or a method?
3. Which state is set for me, and which state am **I** responsible for updating? (In `engine.py` the docstring says this explicitly — most real codebases won't.)
4. What helper methods already exist that I'd otherwise rewrite by hand?
5. Where's the mutation risk — what happens if I run a function twice on the same objects?

Then check yourself: `Request.reserved` is a property (no parens); `Worker.has_room_for()` is a method that already implements the fit check; runtime fields like `finished_at` start as `None` and are yours to set; `fresh()` exists precisely because reuse would otherwise corrupt a rerun.

Habit to carry into the test: before writing any code, write a 5-line map of the given classes. Ten minutes spent here saves thirty later — the Reddit complaints were almost entirely about people skipping this step.

---

7. Wrap-up (5 min)

Do drills c9 and c10 in `day1_drills.py` (the `Task` dataclass and the `Pipeline` class). They're small on purpose — the goal is producing class scaffolding without pausing to recall syntax.

Tomorrow: the timed mock. Read `README.md`, then `engine.py`, then `tests.py` — 10 minutes before touching code.

LLM Inference Engine Primer

A from-scratch guide to the concepts behind the "inference engine" style OA problem: prefill/decode, KV cache, batching, and load balancing. Everything here is simulation-level — no GPUs or ML math needed, just the mechanics an OA would ask you to model.

1. What an inference engine does

When you send a prompt to an LLM server, the server:

1. **Tokenizes** the prompt into a sequence of tokens.
2. **Prefill**: runs the model over *all prompt tokens at once* to build internal state.
3. **Decode**: generates output tokens *one at a time*, each step feeding the previous token back in.
4. Streams tokens back until it hits a stop condition or `max_tokens`.

An "inference engine" (vLLM, TGI, TensorRT-LLM) is the scheduler + memory manager that runs many such requests concurrently on fixed hardware.

2. Prefill vs. decode — the core asymmetry

	Prefill	Decode
Input	Whole prompt (N tokens)	1 token per step
Cost	Proportional to prompt length; compute-bound	Cheap per step; memory-bandwidth-bound
Happens	Once per request	Once per generated token
Latency metric it drives	TTFT (time to first token)	ITL / TPOT (inter-token latency)

Key mental model: **prefill is a big one-shot batch job; decode is a long stream of tiny jobs.** A request's total time = prefill_time(prompt_len) + num_output_tokens x decode_step_time.

In OA simulations this usually becomes something like:

```
prefill_time = ceil(prompt_tokens / prefill_rate)   # e.g. tokens per tick
decode_time  = output_tokens * decode_cost           # e.g. 1 token per tick
```

Watch the units: per-tick rates vs. per-token costs, and whether time is discrete ticks or continuous.

3. KV cache

During prefill, the model computes a key/value tensor for every token; decode steps reuse them instead of recomputing. This is the **KV cache**.

What matters for a simulation:

- KV cache size grows linearly with sequence length (prompt + generated so far).
- It lives in GPU memory, which is finite → this is *the* scarce resource. A GPU can only host as many concurrent requests as its memory allows.
- When a request finishes, its cache is freed.

Typical OA modeling: each request consumes `prompt_len + tokens_generated_so_far` units of a per-GPU memory budget. Admission = only start a request if it fits (sometimes: if its *maximum possible* footprint fits, i.e. `prompt_len + max_new_tokens`).

vLLM's innovation, **PagedAttention**, allocates the cache in fixed-size blocks (like OS virtual memory pages) so memory isn't wasted on over-reservation. An OA might have you allocate/free blocks: `blocks_needed = ceil(seq_len / block_size)`.

4. Batching

GPUs are efficient when processing many requests per step.

- **Static batching**: collect B requests, run them together, wait until *all* finish. Simple, but a short request waits on the longest one.
- **Continuous (in-flight) batching**: at every decode step, the batch is recomposed — finished requests exit immediately, waiting requests join immediately. This is what modern engines do.
- **Chunked prefill**: long prefills are split into chunks and interleaved with decode steps of other requests, so one huge prompt doesn't stall everyone's token stream.

Simulation version of continuous batching, per tick:

```
for tick in count():
    finished = [r for r in running if r.done()]
    for r in finished: free_memory(r); running.remove(r)
    while queue and fits(queue[0]):          # admission policy
        running.append(queue.popleft())
    for r in running: r.step()               # each generates 1 token
```

Edge cases OAs love: what happens on the exact tick a request finishes (does its memory free before or after admission?), tie-breaking among waiting requests (FIFO? shortest-first? priority?), and whether a request arriving at tick t can start at tick t.

5. Scheduling policies

You may be asked to implement one or more of:

- **FCFS / FIFO** — simplest; head-of-line blocking when a long request is first.
- **Shortest-Job-First** — minimizes average latency; needs a job-length estimate (prompt length or max_tokens).
- **Priority scheduling** — requests carry a priority; higher preempts or jumps queue. Define tie-breaks (usually arrival order, then request id).
- **Preemption** — under memory pressure, evict a running request (drop its KV cache), requeue it, and later *recompute* its prefill. Track wasted work.

6. Load balancing across replicas

With multiple GPU workers/replicas, a router assigns each request to a worker:

- **Round robin** — rotate through workers regardless of load.
- **Least connections / least outstanding requests** — send to the worker with fewest active requests.
- **Least load** — same idea but weighted by token counts or memory in use.
- **Session/prefix affinity** — route requests sharing a prompt prefix to the same worker so they can reuse cached KV (prefix caching).

OA gotchas: define "load" exactly as the README does (requests? tokens? memory?), tie-break deterministically (usually lowest worker index), and re-read whether assignment happens at arrival time or at dispatch time.

7. Metrics you might have to compute

- **TTFT** — arrival (or admission?) to first output token.
- **Latency / turnaround** — arrival to completion.
- **Waiting time** — arrival to start of service.
- **Throughput** — tokens (or requests) per unit time.
- **Utilization** — busy ticks / total ticks per worker.

Always check: inclusive or exclusive tick boundaries, and integer vs. float division.

8. Vocabulary cheat sheet

- **Token**: unit of text (~0.75 words).
- **TTFT / TPOT / ITL**: time to first token / time per output token / inter-token latency.
- **KV cache**: stored attention keys/values enabling cheap decode.
- **PagedAttention**: block-based KV cache allocation (vLLM).
- **Continuous batching**: per-step batch recomposition.
- **Chunked prefill**: interleaving prefill pieces with decode.
- **Prefix caching**: reusing KV cache across requests with a shared prompt prefix.
- **Speculative decoding**: small model drafts tokens, big model verifies (unlikely in an OA sim, but know the term).
- **Preemption / eviction**: dropping a running request's cache to free memory.

9. How this maps to the OA

Reported OA shape: a pre-built codebase (classes for requests, workers, maybe a clock), a README describing levels, and functions for you to fill in — e.g. "implement the scheduler step" or "implement the load balancer's pick_worker". The algorithms are easy (queues, heaps, dicts, simulation loops). The difficulty is spec archaeology:

1. **Read the tests first** if visible — they are the real spec. README examples may not match the code; trust the tests, then the class signatures, then the README, in that order.
2. Nail down the **tick model**: what happens within one time step, and in what order (free memory → admit → step? or admit → step → free?). Get this from examples/tests.
3. Nail down **tie-breaking** everywhere a choice exists.
4. Get level 1 passing fast — levels build on each other, and partial credit clearly counts (people advanced with 120/600).

10. Further reading (optional, ~1–2 hrs total)

- vLLM paper/blog on PagedAttention and continuous batching.
- "LLM inference explained" style posts covering prefill/decode and KV cache.
- Orca paper (continuous batching origin) — skim the scheduling section only.

Inference Engine Simulation — Assessment

Implement the functions in `scheduler.py`. Do not modify `engine.py`. Run `python tests.py` to see your score (max **800**, 200 per level). Levels build on each other; partial credit is awarded per test.

Time budget: **90 minutes**. Start the clock before opening `scheduler.py`.

Model

Time advances in integer ticks. A request started at time `s`:

- spends `P = ceil(prompt_tokens / prefill_rate)` ticks in prefill
- then generates exactly one token per tick until `max_new_tokens` tokens have been produced
- its first output token exists at time `s + P + 1`
- it finishes at time `s + P + max_new_tokens`

Level 1 — Sequential worker (200 pts)

Implement `prefill_ticks(prompt_tokens, prefill_rate)` and `run_fifo(requests, config)`.

`run_fifo` simulates a single worker that serves one request at a time, in order of `(arrival, id)`. A request cannot start before it arrives, and cannot start before the previous request finishes.

Example — `prefill_rate=4`, requests `(id=1, arrival=0, prompt=8, max_new=3)`, `(id=2, arrival=1, prompt=4, max_new=2)`:

```
run_fifo(...) == {1: 5, 2: 8}
```

Level 2 — Continuous batching with memory (200 pts)

Implement `run_batched(requests, config)` for a single worker with a KV-cache budget of `config.memory_limit` tokens. Each request reserves `prompt_tokens + max_new_tokens` for its whole lifetime; the reservation is freed when it finishes.

Each tick `t = 0, 1, 2, ...` proceeds in three phases, strictly in this order:

1. **Free**: requests that finished at the end of the previous tick release their reservation.
2. **Admit**: waiting requests are considered in `(arrival, id)` order. Admit the head of the queue while it has arrived (`arrival <= t`) and its reservation fits. Stop at the first request that has arrived but does not fit (head-of-line blocking: later requests may NOT jump the queue).
3. **Work**: every running request advances one tick — `prefill_rate` prompt tokens of prefill, or one generated token if prefill is complete. A tick that completes prefill does not also generate a token.

Return a dict mapping `id -> (first_token_at, finished_at)`.

Level 3 — Load balancer (200 pts)

Implement `pick_worker(loads)` and `route(requests, config, num_workers)`.

`pick_worker(loads)` returns the index of the worker with the least load, breaking ties toward the lowest index.

`route` assigns requests to workers at arrival, in `(arrival, id)` order. A worker's load is the total `reserved` tokens of all requests assigned to it so far (assignments are permanent — no rebalancing). Each worker then runs its assigned requests independently, exactly as in Level 2.

Return `(assignment, results)` where `assignment` maps `id -> worker index` and `results` maps `id -> (first_token_at, finished_at)`.

Level 4 — Preemption & priorities (200 pts)

Implement `run_preemptive(requests, priorities, config)`. Single worker. `priorities` maps `id -> int`; **higher means more urgent**.

Changes from Level 2:

- Admission order is `(-priority, arrival, id)` — there is **no** head-of-line blocking; any arrived request may be considered.
- If the best candidate does not fit, **evict** running requests whose priority is *strictly lower* than the candidate's, until it fits or no such victim remains. Victim choice: lowest priority first, ties broken by **highest id**.
- An evicted request loses all progress (prefill must be redone) and returns to the queue with its original `arrival`. It may **not** be re-admitted on the same tick it was evicted.
- Repeat admission until no further request can be admitted this tick.

Return `(results, eviction_count)` where `results` is as in Level 2.

Notes

- Determinism matters: all ties are broken by lowest id / lowest index.
- The graders instantiate fresh `Request` objects per test; you may mutate the ones you're given.

SPOILERS — read only AFTER attempting the mock

Built-in traps (deliberate, mirroring the real OA complaints)

1. **README vs. code mismatch**: the Level 1 README example shows a dict (`{1: 5, 2: 8}`) but `run_fifo`'s docstring — and the tests — expect a **list of tuples in completion order**. Lesson: when README and code disagree, trust tests > signatures/docstrings > README.
2. **Tick-order pedantry**: Level 2's free → admit → work order changes answers. A request finishing "at the end of tick t" frees memory at the start of tick t+1, so a waiting request admitted then starts prefill on tick t+1, not t.
3. **Prefill-completion tick generates no token** — classic off-by-one.
4. **Head-of-line blocking** in admission: a small request that fits may NOT skip a big one ahead of it in the queue.
5. **`Request.fresh()`** exists because runtime state is mutable — if you run a simulation twice on the same objects you get garbage. The engine file hints at this; noticing it is part of the exercise.
6. **Level 4 reverses a Level 2 rule**: head-of-line blocking is gone, and admission order changes. Real OAs do this — a later level invalidates an assumption you baked in. Write Level 2 so the admission policy is easy to swap, and re-read each level's spec instead of assuming continuity.
7. **Eviction determinism**: victim = lowest priority, ties by *highest* id (not lowest — read carefully). Evicted requests lose prefill progress and can't be re-admitted the same tick, which prevents livelock.

How to grade yourself

- `python tests.py` — your score.
- `python tests.py --solution` — verify the reference gets 800/800.
- 500+ in 90 minutes: strong. 300–500: fine — focus on whichever level broke. Levels 1–3 fully solved (600/800) is a solid outcome; level 4 is deliberately the time sink.

Debrief questions

- How long before you wrote your first line of code? (Target: ≤15 min of reading, including tests.)
- Did you read `tests.py` before implementing? You were allowed to. The expected values there resolve every ambiguity in the README.
- Which trap cost you the most time?

Reference solution

`solution/scheduler_solution.py`. To rerun the mock later, restore `scheduler.py` stubs and try again with different self-imposed time limits.